a void return type, then we label this method as a *setter*. Table 1 shows the number of getters/setters for each project.

Controlling for Age: The Kendall's τ correlation coefficient between ages of methods and their number of revisions is 0.2 in our aggregated dataset. When considered individually, some projects show very high correlation (e.g., 0.51 for the checkstyle project). Therefore, when analyzing the relationship between SLOC and maintenance indicators, we should not include methods with different ages. Generally speaking, a ten year old method will have more chances to change than a newly introduced method, regardless of their code complexity. To perform age normalization, we followed a two step procedure. In step 1, we removed methods that are less than two years old. This does not, however, solve the problem completely, because we may still compare a 2 years old with a $2{+}x$ years old method. Therefore, in step 2, we excluded all revisions and bugs that happened after two years of a method's life. If we set the threshold more than two years, we lose many methods from our dataset. As presented in Table 1, with two year threshold, we still have 520,874 methods for our final analyses. If we set the threshold less than two years, we lose many change and bug information, because the window to observe changes becomes narrower.

Multiple Versions: A method can have multiple SLOCs while it evolves. Consider a method associated with 5 SLOCs in its lifetime: 20, 50, 50, 50, 22. The method was revised 4 times after its introduction, so the value of #revisions is 4. To calculate the correlation between SLOC and #revisions, what SLOC value should represent this method? If we take the introduction SLOC, we will map SLOC 20 to 4 revisions, which is inaccurate because the method was revised only once for SLOC 20. We solve this problem by applying a versioning technique, where the above method has three different versions. That way version 1 has SLOC 20 with 1 revision, version 2 has SLOC 50 with 3 revisions, and version 3 has SLOC 22 with 0 revision, because the method did not change afterwards. We apply this technique for other maintenance indicators as well.

3.6 Statistical Tests

Throughout this paper, we compare if different distributions are statistically different. We applied the Anderson-Darling normality test [83] to some of the randomly selected distributions and found that many of the distributions are not normally distributed. Therefore, non-parametric tests are selected for our statistical testing. For evaluating if two distributions are different, we use the Wilcoxon rank sum test (and Wilcoxon signed rank test for pairwise comparison). For calculating the correlation coefficient between two variables, we use the non-parametric Kendall's τ, instead of the Pearson's correlation. For calculating effect sizes, we apply Cliff's Delta instead of Cohen's d. In accordance with [34], we classify an effect size either as Negligible ($0 \leq |\delta| < 0.147$), Small ($0.147 \leq |\delta| < 0.330$), Medium ($0.330 \leq |\delta| < 0.474$), or Large ($0.474 \leq |\delta| \leq 1$). These non-parametric tests are widely used in software engineering research [19, 26, 37].

4 Results

In this section, we present our findings to our four research questions. We investigate whether method size is an important factor to understand change- and bug-proneness, what the upper-limit of method's size should be, whether developers should decompose large methods into smaller ones, and if controlling for size controls other code quality factors.

4.1 RQ1: Is SLOC correlated with change- and bug-proneness at method-level?

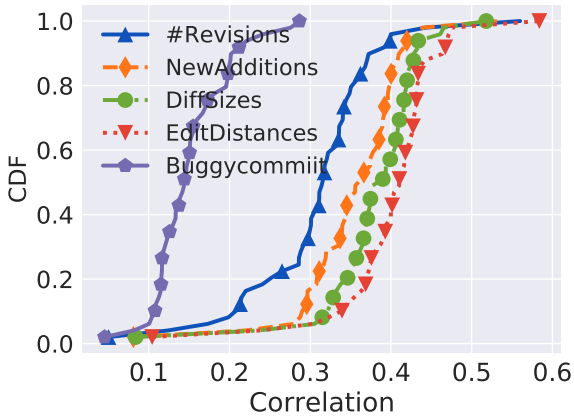

Figure 1: Cumulative distribution functions of the correlation between SLOC and the five maintenance indicators. Each line represents 49 points for our 49 different projects. For graph readability, number of markers has been reduced.

Figure 1 shows the distribution of Kendall's τ correlation coefficients between SLOC and the five maintenance indicators. Correlation coefficient for each project is shared with our pubic dataset as a table as well (`stats/table-2.pdf`). For all projects, the correlation coefficients are positive, indicating method size is an indicator of future maintenance. When compared among the four change-proneness indicators, SLOC performance is generally the lowest for #revisions, and highest for EditDistance. Correlations between SLOC and bug-proneness (i.e., #BuggyCommits) are significantly lower than the four change-proneness indicators. This implies that, at the method-level, code metrics are comparatively less helpful for bug prediction than for change prediction. This complements the recent findings of Pascarella *et al.* [62], who consider method-level bug prediction as an open research problem. We argue that it is unrealistic to expect very high correlation between a code metric and maintenance effort. A very high correlation would mean that maintenance effort can be estimated just by using one code metric. This is unrealistic because there are many factors that influence code maintenance, such as developer habits [82], application domain and platforms [88, 92], code clones [58], software architecture [4], test code quality [9, 78], and changes in requirements.

We also observe that SLOC performance is not similar across projects. For the docx4j project, SLOC correlates to change-proneness very weakly (e.g., correlation is only 0.05 with #revisions) while for the voldemort project the correlation is 0.56. This observation is true for bug-proneness as well. For the cucumber-jvm project, the correlation is only 0.05, but for the xerces2-j, the correlation is 0.29. These observations explain why cross project maintenance prediction is still a challenging problem [81, 92].